Technische Universität München

Übungen Rechnereinführung WS 2026/27

Adrian Stein – Thomas Lindae – Ari Breitkreuz

---

1 – Linux (Teil 1)

Aufgaben, welche mit einem (*) markiert sind, sind Bonusaufgaben und sollten nur bearbeitet werden, wenn man die Zeit dafür hat.

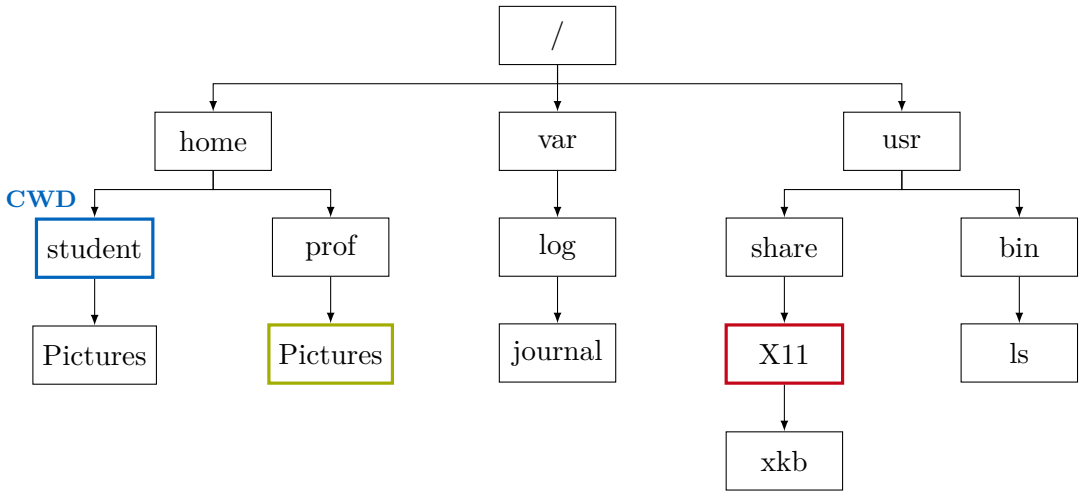
1.1 Hinweis

Wie auch schon in der Vorlesung besprochen, ist das Ziel nicht, dass Sie jedes Kommando auswendig kennen mit allen möglichen Optionen. Auf diesem Blatt wollen wir den Umgang mit den wichtigsten Kommandos und Funktionen im Terminal zu übern und zu lernen, wie Sie sich selber helfen können durch z. B. das Linux Manual (bzw. das zugehörige Kommando `man`).

1.2 Das Linux-Filesystem

1.2.1 Filesystem Pfade

Es sei der folgende Ausschnitt an Verzeichnissen in einem Linuxsystem gegeben:



Geben Sie den Pfad zum jeweils grün und rot hervorgehobenen Ordner an als:

1. Absoluten Pfad
2. Relativen Pfad
3. Pfad, der ein `~` benutzt, wenn man als der Benutzer „prof“ eingeloggt ist

Lösung

Grün:

1. `/home/prof/Pictures`
2. `../prof/Pictures`
3. `~/Pictures`

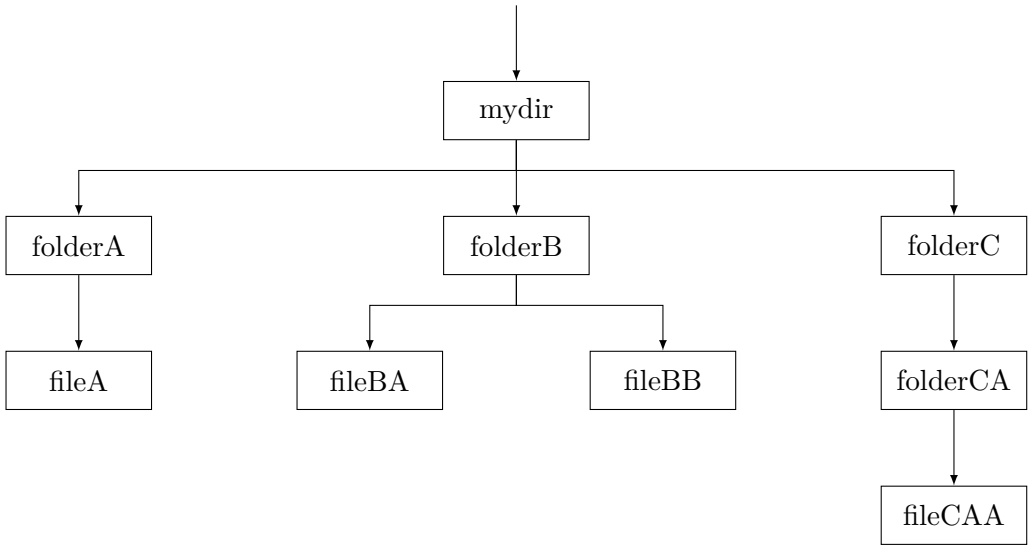
Rot:

1. `/usr/share/X11`
2. `../../usr/share/X11`
3. `~/../../usr/share/X11`

Case sensitivity beachten!

1.2.2 Ordner erstellen mit `mkdir`

1. Erstellen Sie in einem beliebigen Ordner mithilfe von `mkdir` und `touch` die folgende Struktur:



2. Lesen Sie sich den `man` Eintrag zu `mkdir` durch. Wie können Sie den Ordner `~/uni/vorkurs/rechner` mit **genau einem** Kommando erstellen?
3. (*) Führen Sie erneut das gleiche Kommando aus aber lassen Sie sich ausgeben, welche Ordner dabei erstellt werden. Benutzen Sie nur ein `-` im Kommando.

Lösung

1. Sehr leicht zu überprüfen mit `tree`.
2. `mkdir -p Documents/uni/vorkurs` (Ausschlaggebend ist hier das `-p`)
3. `mkdir -pv Documents/uni/vorkurs`. Äquivalent wäre `--parent --verbose`, aber es soll nur ein `-` benutzt werden.

1.2.3 Navigation

Benutzen Sie ein paar mal hintereinander `cd` mit verschiedenen Zielverzeichnissen. Probieren Sie nun herauszufinden, was das Kommando `cd -` macht.

Lösung

`cd -` bringt einen zurück zum vorherigen Verzeichnis.

1.2.4 Ordnerinhalte und versteckte Dateien mit `ls`

Lesen Sie sich den `man` Eintrag zu `ls` durch.

1. Wie werden in Linux Dateien „versteckt“? Wie kann man solche Dateien anzeigen?
2. Wird im Root-Verzeichnis `..` angezeigt? Wie kann man das herausfinden, ohne das aktuelle Verzeichnis zu verlassen?
3. Listen Sie **alle** Dateien im aktuellen Verzeichnis (mit allen Informationen), bis auf `.` und `..`, färbe Verzeichnisse ein und sortiere nach neuster Zugriffszeit.

Lösung

1. „Versteckte“ Dateien fangen mit einem Punkt an. Man kann sie mit `ls -a` ausgeben
2. Ja, benutze das Kommando `ls -a /`
3. `ls -Altu --color=auto` oder `ls -A -l -t -u --color=auto` (oder beliebige permutation der Flags)

1.2.5 (*) Timestamps und `touch`

Lesen Sie sich den `man` Eintrag zu `touch` durch.

1. Erstellen Sie eine Datei `sehr.alt`, welche das Modifikationsdatum 01.01.1990 hat. Benutzen Sie `ls`, um Ihr Ergebnis zu überprüfen.
2. Verändern Sie nur das Modifikationsdatum der Datei zur aktuellen Zeit.

Lösung

1. `touch -t 199001010000 very.old`
2. `touch -m very.old`

1.2.6 Löschen mit `rm`

Lesen Sie sich den `man` Eintrag zu `rm` durch.

1. Ist es möglich eine Datei, die durch `rm` gelöscht wurde, wiederherzustellen?
2. Was macht `shred` anders als `rm`? Können Dateien die mit `shred` gelöscht wurden wiederhergestellt werden?

3. (*) Im Internet gibt es den Witz, `rm -rf /` auszuführen, welches das komplette Betriebssystem löschen würde. Welchen Schutz bietet `rm` gegen dieses Kommando?

Lösung

1. Es gibt keinen „Papierkorb“, aus dem man die Dateien leicht wiederherstellen könnte. Dennoch werden die Dateien **nicht** auf der Festplatte zerstört. Mit genug Aufwand kann man diese also retten.

   Eine Demo für *natives* Linux:

   Um das ganze einmal zu veranschaulichen kann am besten auf einem USB Stick eine sehr kleine Partition (ein paar MB) erstellt werden (ist die Partition größer, wird die Suche wesentlich länger dauern). Auf dieser Partition muss nun eine Datei (z. B. `leahcim.txt`) mit beliebigen ASCII text gespeichert werden (z. B. „regrebnettuL“, `echo "regrebnettuL" >> leahcim.txt`). Da der Linux Kernel die Dateien u. U. nicht sofort auf die Platte schreibt, muss bevor mit der Demo fortgefahren wird ein manueller *Filesystem-Sync* durchgeführt werden. Entweder mit `sync -f path/to/drive-mountpoint` oder einfach indem der Stick einmal ausgeworfen wird. Anschließend kann die Datei gelöscht werden `rm leahcim.txt`. Für die Recovery muss zunächst das „device“ der Demo-Partition gefunden werden (einfach mit `lsblk`. Die Partition kann z.B. in `/dev/sdd1` liegen). Nun kann mit `sudo strings /dev/sdd1 | grep "regrebnettuL"` nach dem Inhalt der gelöschten Datei gesucht werden (`strings` gibt uns einfach nur alle möglichen Strings mit einer mindestlänge von 4 die auf der Partition vorhanden sind). Das ganze funktioniert weil über `/dev/<some-drive>` das Filesystem umgangen wird und direkt die Bytes auf den angeschlossenen Festplatten betrachtet werden kann.

2. `shred` versucht den Inhalt einer Datei vor dem löschen durch mehrfaches überschreiben obfuskieren, um so die Recovery massiv zu erschweren. Ob dies gelingt wird jedoch nicht allgemein garantiert! Zusätzliche recovery Maßnahmen des Filesystems oder die Speicher-Hardware selbst können u. U. den Effekt von `shred` nichtig machen (siehe `man shred`)!

3. Man müsste noch das `--no-preserve-root` Flag benutzen. (Über den Schutz von `rm` hinaus fehlt auch `sudo`)

1.3 Control & Redirection Operators

In der Vorlesung haben Sie bereits die Control Operators `||`, `&&`, `;`, `&`, `()` und `|` (auch „Pipe“ genannt), sowie die Redirection Operators `>`, `>>`, `<` und `<<` kennengelernt. In den

folgenden Aufgaben wird der Umgang mit diesen Operatoren geübt.

1.3.1 Pipes und Ausgabeumleitung

Auf Moodle finden Sie neben diesen Übungsblättern auch die folgenden beiden Text Dateien: `study1.txt` und `study2.txt` (beide Dateien enthalten ein Subset der von der TUM angebotenen Studiengänge). Für diese Aufgabe sind die Kommandos `sort` und `uniq` hilfreich (überfliegen Sie bei bedarf die zugehörigen `man` Einträge).

1. Was ist der Unterschied zwischen `>>` und `>`? Leiten Sie die Ausgabe von `echo` mit diesen Operatoren in Test-Dateien und finden Sie es experimentell heraus.
2. Was ist der technische Unterschied `2>>` und `>>` (bzw. `2>` und `>`)?
   (*Hinweis: Leiten Sie zum testen die Ausgabe eines Programms um welches u. U. fehlschlagen kann*)
3. Geben Sie den Inhalt beider Dateien mit nur einem Aufruf von `cat` auf der Konsole aus.
4. Können Sie den Aufruf `cat` optimieren um eine vielzahl von Dateien ohne große schreibarbeit auszugeben? Informieren Sie sich diesbezüglich im Internet über „globbing“.
5. Geben Sie zwei Kommandos an, mit welchem man den Inhalt von `study1.txt` und `study2.txt` jeweils sortiert ausgeben kann.
6. Eleminieren Sie nun in der sortierten Ausgabe beider dateien zusätzlich alle Duplikate, ohne die Dateien zu verändern.
7. Geben Sie in einem Kommando den Inhalt beider Dateien sortiert und frei von Duplikaten aus. Wie könnte man nun diese Ausgabe in einer neuen Datei Namens `study-all.txt` speichern?
8. Fügen Sie nun der erstellten Datei `study-all.txt` den Studiengang „Medizin“ hinzu (ohne einen Text Editor zu benutzen). Sortieren Sie anschließend die Datei erneut.
9. Geben Sie nun jeweils die ersten und letzten 4 Zeilen der `study-all.txt` aus (Die Kommandos `head` und `tail` sind womöglich hilfreich).
10. Lassen Sie eine Kuh mit ausgestreckter Zunge (`" U"`) ein Haiku mithilfe eines *Here-Documents* aufsagen.

Lösung

1. `>>` fügt an das Ende der Datei hinzu, `>` ersetzt den Inhalt der Datei
2. Es werden nur Fehlermeldungen umgeleitet. Die Ziffer beschreibt welcher

Dateidescriptor für die Umleitung benutzt werden soll. Per default wird für normale Ausgaben 1 benutzt, was *stdout* entspricht. Fehlermeldungen werden über 2 (*stderr*) ausgegeben. Das Terminal zeigt standardmäßig Ausgaben von beiden an.

3. `cat study1.txt study2.txt`
4. `cat *.txt`, `*.txt` meint in diesem Fall alle Dateien die in `.txt` enden im aktuellen *CWD*
5. `sort study1.txt` bzw. `sort study2.txt`
6. `sort study1.txt | uniq` bzw. `sort study2.txt | uniq`
7. `cat study1.txt study2.txt | sort | uniq` für nur die Ausgabe und `cat study1.txt study2.txt | sort | uniq > study-all.txt` um die Ausgabe in einer neuen Datei zu speichern.
8. `echo "Medizin" >> study-all.txt`, anschließend `sort study-all.txt -o study-all.txt` (**Achtung:** `sort study-all.txt > study-all.txt` geht nicht weil die Datei zuerst *truncated* wird und somit `sort` einen leeren input hat).
9. `cat study-all.txt | head -n 4` und `cat study-all.txt | tail -n 4`
10.
```
cowsay -T " U" <<EOF
Squish, squash, sling that slang.
I'm always right back at ya,
like my... boomerang!
EOF
```

1.3.2 Verknüpfte Kommandos

In dieser Aufgabe werden Operatoren (logisch) miteinander verknüpft. Schreiben Sie jeweils ein Verküpfung von Kommandos die:

1. In den Ordner `test` wechselt. Falls der Ordner nicht existiert soll auf der Konsole zusätzlich „`== Error ==`“ ausgegeben werden.
2. In den Ordner `test` wechselt und anschließen den Inhalt des Ordners, inklusiver versteckter dateien, auflistet (vorausgesetzt der wechsel hat funktioniert).
3. Im Hintergrund läuft und drei mal alle 5 Sekunden „`TUM`“ ausgibt (hier ist das Kommando `sleep` hilfreich).

Lösung

1. `cd test || echo "== Error =="`
2. `cd test && ls -a`
3. `( echo "TUM"; sleep 5; echo "TUM"; sleep 5; echo "TUM" ) &`